

TP N°1: arVault

[75.73/TB034] Arquitectura de Software
CURSO: 01-Calónico
Primer cuatrimestre de 2026

Alumno/a:	FARALL CASADO, Tomás
Número de padrón:	102628
Email:	...

Alumno/a:	VACCARELLI, Santiago
Número de padrón:	106051
Email:	svaccarelli@fi.uba.ar

Alumno/a:	JANG, Lucas
Número de padrón:	109151
Email:	ljang@fi.uba.ar

Alumno/a:	KRESTA, Facundo Ariel
Número de padrón:	110857
Email:	facundoarielkresta@gmail.com

Índice

1	Introducción	2
1.1	Contexto	2
1.2	Objetivo	2
2	Atributos Importantes	3
2.1	Atributos de Calidad Clave	3
2.2	Crítica a las Decisiones de Diseño Originales	3
2.3	Desempeño y Pruebas de Carga del Caso Base	4
3	Decisiones de Diseño (C&C)	5
3.1	Evolución: Del Monolito Acoplado a Arquitectura en Capas	5
3.1.1	Mejora propuesta	5
3.1.2	Atributos de calidad afectados	7
3.2	Variables en memoria	9
3.2.1	Mejora propuesta	10
3.2.2	Atributos de calidad afectados	10
3.3	Persistencia lazy	10
3.3.1	Mejora propuesta	10
3.3.2	Atributos de calidad afectados	10
3.4	Persistencia en archivos json	10
3.4.1	Mejora propuesta	11
3.4.2	Atributos de calidad afectados	11
3.4.3	Función transfer() mockeada de forma secuencial	11
3.5	Instancia única sin redundancia ni healthcheck	11
3.5.1	Mejora propuesta	11
4	Métricas: Caso Base (Lectura de Rates)	14
5	Conclusión (Parcial)	16

1. Introducción

1.1. Contexto

La startup **arVault** es una fintech que opera una billetera digital, de reciente creación. Su fundador, un desarrollador aficionado y entusiasta con algo de dinero y muchas ideas, consciente de que la clave de su negocio es la implementación de su **backend**, tercerizó el desarrollo de las aplicaciones para dispositivos móviles, y se concentró en implementar rápidamente (bajo la consigna *fake it till you make it*¹) un núcleo de servicios que le permitieran conseguir más fondos a través de inversores.

Luego de la ronda inicial, los primeros fondos fueron utilizados, no para robustecer la arquitectura existente, sino para agregar más funcionalidad. Una de estas funcionalidades nuevas permite abrir cuentas en distintas monedas (que se respaldan en bancos existentes), y realizar operaciones de cambio entre éstas. El diferencial de **arVault** es tener la tasa de cambio más conveniente dentro de las aplicaciones que proveen este servicio. Para ganar usuarios, las tasas de cambio no tienen *gap* entre el valor de compra y de venta.

Si bien el lanzamiento fue exitoso, comenzaron a aparecer reclamos de los usuarios sobre problemas en el uso de la función de cambio de monedas. Estos reclamos llegaron en el peor momento, dado que **arVault** necesita captar más inversiones y, debido a la caída de la reputación y las reviews negativas, los potenciales inversores se niegan a aportar más fondos si no se realiza una auditoría y se mejora el servicio.

Frente a este reclamo, **arVault** decide contratar a un grupo de arquitectos para que evalúen, propongan e implementen soluciones que mejoren los atributos de calidad del servicio de cambio de monedas.

1.2. Objetivo

El objetivo de este informe es presentar una evaluación de la arquitectura actual del servicio de cambio de monedas, identificar los atributos de calidad que se ven afectados (ver Apartado 2) por los reclamos de los usuarios, y proponer decisiones de diseño que puedan mejorar dichos atributos (Apartado 3). Además, se presentarán métricas en el Apartado 4 para evaluar el impacto de las decisiones propuestas y se discutirán las conclusiones obtenidas a partir del análisis realizado. Finalmente, se incluirá una sección de conclusiones (Apartado 5) que sintetice los hallazgos y recomendaciones para **arVault**.

¹Expresión popular que refiere a simular confianza o habilidades hasta realmente adquirirlas.

2. Atributos Importantes

2.1. Atributos de Calidad Clave

Considerando la naturaleza financiera y transaccional del servicio provisto (*arVault*, un *exchange* de criptomonedas y dinero fiat), se han determinado como centrales los siguientes Atributos de Calidad (QAs):

- **Seguridad (Security):** Al tratarse de una *fintech* que opera saldos reales en múltiples monedas, la seguridad es el atributo mas critico. Si bien existe un componente externo que gestiona autenticación y autorización, hay aspectos de seguridad que recaen directamente sobre el servicio: la integridad de las transacciones, la prevención de condiciones de carrera que permitan operar con fondos insuficientes y la consistencia del estado financiero. Una brecha en este atributo tiene consecuencias legales, financieras y reputacionales inmediatas.
- **Disponibilidad (Availability):** arVault ya sufrió caída de reputación y pérdida de inversores por problemas en el servicio de cambio. Para una billetera digital, la indisponibilidad equivale a pérdida de negocio y confianza de los usuarios. Este atributo es condición necesaria para la continuidad operativa de la empresa.
- **Desempeño (Performance):** El diferencial de arVault es ofrecer la tasa de cambio más conveniente del mercado. Si las operaciones de *exchange* presentan latencias elevadas, los usuarios perciben el servicio como poco confiable, especialmente en un contexto donde los tipos de cambio fluctúan en tiempo real. El sistema debe procesar múltiples operaciones concurrentes con un tiempo de respuesta adecuado y un alto nivel de *throughput* bajo momentos de mayor demanda.
- **Escalabilidad (Scalability):** La startup se encuentra en crecimiento activo, captando usuarios e inversores. La arquitectura original fue construida bajo la consigna *fake it until you make it* y no fue diseñada para escalar. Previendo un escenario con cantidad creciente de usuarios operando simultáneamente, el servicio debe poder ser replicado horizontalmente para poder soportar la carga aumentada de usuarios.
- **Usabilidad (Usability) — tercerizado:** El fundador de arVault tercerizó el desarrollo de las aplicaciones móviles, por lo que la usabilidad del frontend queda fuera del alcance directo de este análisis. Se menciona por su importancia, pero no es objetivo de las tácticas a implementar en este trabajo.

2.2. Crítica a las Decisiones de Diseño Originales

La arquitectura y código del “caso base” presentan decisiones de diseño cuestionables desde el punto de vista arquitectónico, las cuales impactan drásticamente en los QAs:

- **Manejo del Estado y Confiabilidad (Reliability):** El módulo `state.js` carga inicialmente todo el estado a memoria desde archivos locales JSON y emplea un ciclo `setInterval` cada 1 a 5 segundos (`scheduleSave`) para reescribir el archivo entero de forma asincrónica. Si el contenedor o el proceso muere de manera abrupta, las operaciones (*exchanges*) realizadas desde la última escritura hasta el instante de la falla se pierden irremediamente, rompiendo la confiabilidad del servicio y dejando perfiles inconsistentes.
- **Seguridad (Security):** En el código de `exchange.js`, el diseño de la asincronía introduce condiciones de carrera (*race conditions*) que comprometen la integridad financiera del sistema. Al evaluar `counterAccount.balance >= counterAmount` y luego bloquear el hilo con `await transfer(...)`, múltiples peticiones concurrentes pueden superar la validación de saldo simultáneamente antes de que se actualicen los balances. El resultado es que el sistema permite ejecutar *exchanges* con fondos insuficientes, violando reglas de dominio críticas de consistencia y seguridad.
- **Escalabilidad:** El estado dependiente de la memoria de la aplicación en nodo y la escritura de archivos simples dentro del propio contenedor, rompe uno de los pilares de arquitecturas en la nube (stateless). Al no poseer un mecanismo de sincronía centralizado (como una base de datos ACID), es imposible desplegar y sincronizar múltiples instancias de la aplicación (escalamiento horizontal) tras un balanceador de carga. Cada instancia trabajará con un estado divergente de balances.

- **Testabilidad (Testability) y Modificabilidad:** El alto acoplamiento y el almacenamiento manual limitan fuertemente la escritura de pruebas aisladas, limitándose el sistema a un escenario fraccionario.

2.3. Desempeño y Pruebas de Carga del Caso Base

Para evidenciar el comportamiento de los atributos de **Performance** sobre la arquitectura base, se vuelve indispensable contar con métricas precisas. Tal como se puede apreciar en la Sección 4, al ejecutar una prueba de carga sobre el endpoint de consultas (**GET /rates**), el sistema denota una latencia mediana excelente (~ 3 ms, $p99$ de 13,1 ms) y una tasa de respuesta estable, ya que es una mera consulta a la memoria (operación no bloqueante y sin I/O pesado).

Aun así, este comportamiento favorable es dependiente de que las operaciones no transaccionen (como **/exchange**).

3. Decisiones de Diseño (C&C)

3.1. Evolución: Del Monolito Acoplado a Arquitectura en Capas

Originalmente, el servicio constaba de un monolito de una sola capa. Había tres archivos con roles distintos (`app.js` operando de HTTP handler, `exchange.js` manejando lógica de negocio, y `state.js` interactuando con la persistencia en disco) que se encontraban acoplados directamente sin una clara inyección de dependencias ni definición de interfaces seguras. Todo corría en el hilo de un único proceso de Node.js, penalizando *Testability*, *Maintainability* y *Modifiability*. El caso base se ilustra debajo:

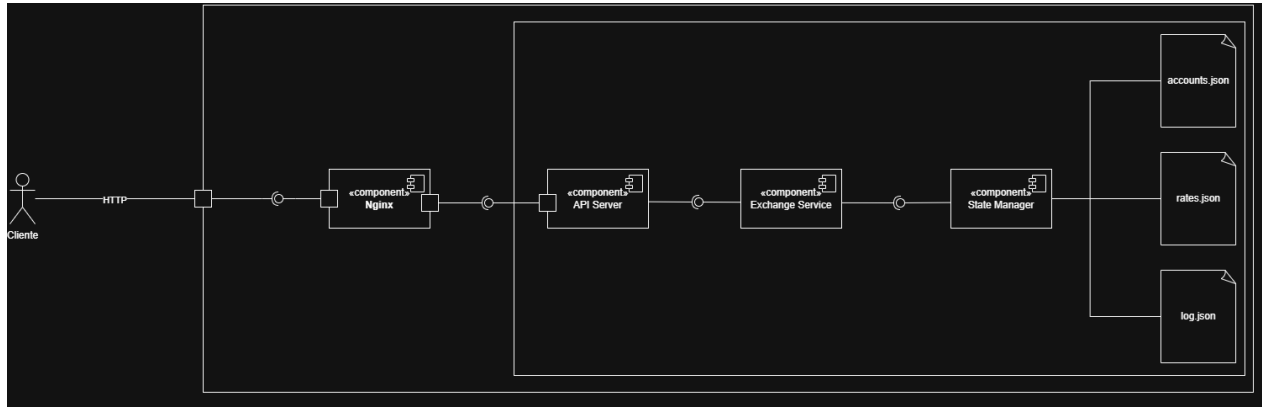


Diagrama 3.1: Arquitectura monolítica del sistema

El **Exchange Service** accede directamente a referencias internas del **State Manager** y muta su estado sin pasar por ninguna interfaz definida, *bypaseando* por completo los métodos expuestos por el **State Manager**.

Además, el **API Server** mezcla responsabilidades: realiza validación de input, invoca la lógica de negocio y decide el formato de respuesta todo en el mismo handler, sin delegar al **Exchange Service** de forma limpia.

3.1.1. Mejora propuesta

La arquitectura propuesta introduce una separación en capas con responsabilidades bien delimitadas. El **API Server** delega cada request a uno de cuatro controladores especializados: **Account Controller**, **Rate Controller**, **Exchange Controller** y **Log Controller**, que se encargan exclusivamente de la validación de input y el mapeo de respuestas HTTP. Cada controlador invoca a su correspondiente servicio de dominio (**Account Service**, **Rate Service**, **Exchange Service** y **Log Service**), donde reside la lógica de negocio. Los servicios acceden al estado únicamente a través de repositorios (**Account Repository**, **Rate Repository**, **Log Repository**), que exponen interfaces limpias y son los únicos componentes autorizados a interactuar con el **State Manager**. Este último centraliza toda la lectura y escritura a disco, persistiendo el estado en `accounts.json`, `rates.json` y `log.json`.

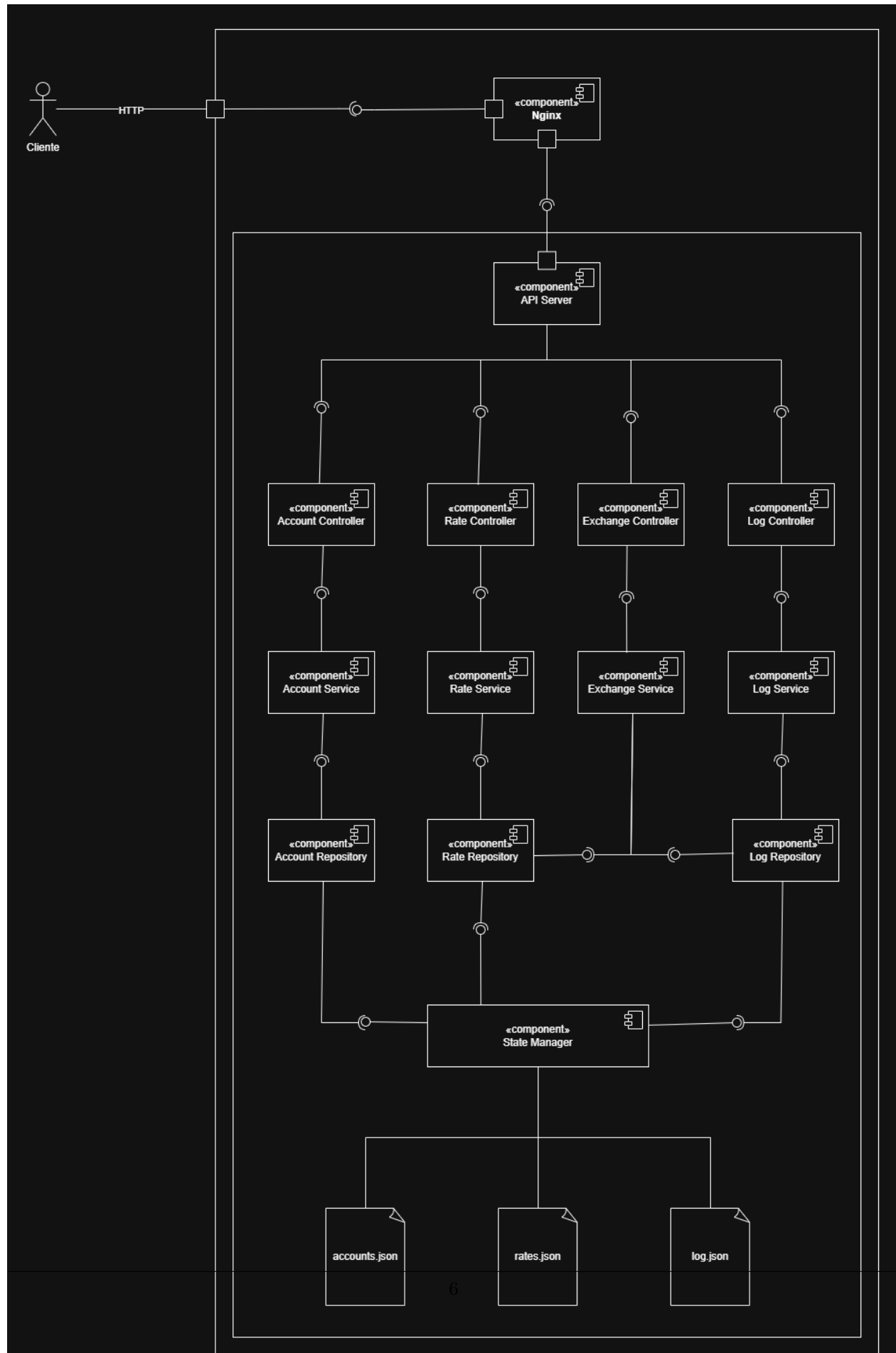


Diagrama 3.2: Arquitectura luego de la mejora

3.1.2. Atributos de calidad afectados

Testability Es el atributo más beneficiado por este cambio. En la arquitectura original, testear el **Exchange Service** implicaba activar toda la cadena de componentes, ya que no había separación entre lógica de negocio, acceso a datos y manejo de estado. En la nueva arquitectura, cada capa puede probarse de forma aislada: los **Controllers** pueden testearse con mocks de los **Services**, los **Services** con mocks de los **Repositories**, y los **Repositories** con mocks del **State Manager**. Esto permite pruebas unitarias verdaderas y facilita la detección temprana de bugs.

Maintainability También mejora considerablemente. La separación por capas y por responsabilidades hace que encontrar a nivel de código dónde está cada cosa sea más fácil e intuitivo, sin tener que recorrer código ajeno a lo que se busca. Si surge un bug, el patrón establecido lo hace directo de encontrar; cada feature nuevo sigue la misma estructura.

Modifiability El cambio tiene un impacto positivo significativo. Con la arquitectura anterior, agregar un nuevo dominio o modificar uno existente podía requerir alterar componentes compartidos como el **Exchange Service**, propagando el riesgo de cambio por toda la cadena. En la nueva arquitectura, cada dominio es una columna vertical independiente (**Controller** → **Service** → **Repository**), de modo que agregar un nuevo dominio implica simplemente agregar una nueva columna sin tocar las existentes. Del mismo modo, cambiar la fuente de datos de los logs, por ejemplo, solo requiere modificar el **Log Repository**, sin afectar al **Log Service** ni al **Log Controller**. Esto reduce el costo y el riesgo de cada modificación futura.

Performance (atributo afectado negativamente) Si bien en teoría la nueva arquitectura introduce más niveles de indirección y por lo tanto podría esperarse un impacto negativo en la latencia, en la práctica las métricas no mostraron diferencias significativas respecto a la arquitectura anterior, o las diferencias registradas fueron lo suficientemente pequeñas como para considerarse despreciables. El overhead generado por los saltos adicionales entre capas no se tradujo en una degradación observable del sistema.

3.2. Variables en memoria

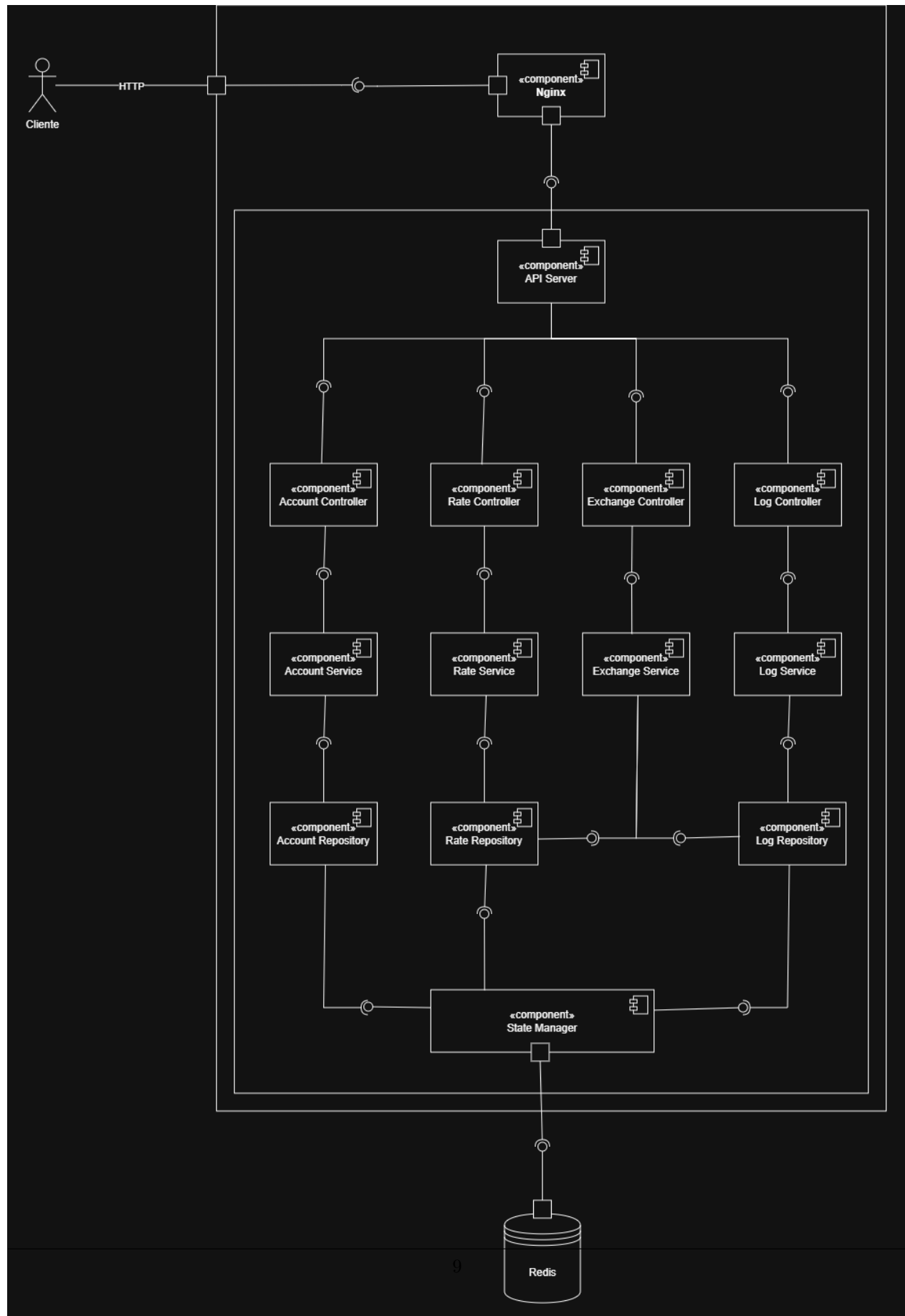


Diagrama 3.3: Arquitectura con Redis como almacenamiento centralizado. Las tres decisiones de diseño que siguen —variables en memoria, persistencia lazy y persistencia en archivos JSON— fueron resueltas todas juntas mediante la incorporación de Redis. El **State Manager** ya no persiste en archivos locales sino que delega en Redis, que actúa como fuente de verdad única y compartida entre todas las instancias.

Accounts, rates y log son variables JavaScript dentro del proceso. Leer es instantáneo (bueno para *performance*), pero hace imposible escalar horizontalmente porque cada instancia tendría su propio estado desincronizado. Además genera una *race condition*: dos requests concurrentes pueden leer el mismo balance, pasar ambos el check de fondos, y producir doble gasto.

3.2.1. Mejora propuesta

Dado que el estado en memoria hacía imposible la escalabilidad horizontal y generaba una *race condition* que podía derivar en doble gasto, se decidió incorporar Redis como almacenamiento centralizado. De esta forma el estado deja de vivir dentro del proceso y pasa a una fuente de verdad única y compartida entre todas las instancias, eliminando la desincronización y permitiendo operaciones atómicas.

3.2.2. Atributos de calidad afectados

Scalability El atributo más impactado. El estado deja de vivir dentro del proceso y pasa a un almacenamiento externo compartido. Antes era imposible levantar múltiples instancias porque cada una tendría su propio estado desincronizado; con Redis esto queda resuelto.

Reliability La *race condition* del doble gasto desaparece porque Redis permite operaciones atómicas, de modo que dos requests concurrentes ya no pueden leer el mismo balance y pasar ambas el check de fondos simultáneamente.

Performance (atributo afectado negativamente) Leer una variable en memoria es instantáneo, mientras que cada operación contra Redis implica una llamada de red. En la práctica esta latencia suele ser de un par de milisegundos y resulta aceptable, pero es un costo real que antes no existía.

3.3. Persistencia lazy

El estado se guarda a disco cada 1-5 segundos en lugar de en cada operación. Simplifica el código y no bloquea requests con I/O, pero si el proceso muere entre guardados se pierden transacciones, violación de durabilidad.

3.3.1. Mejora propuesta

Con la incorporación de Redis, decidimos remover la persistencia lazy a disco. Redis se encarga de la durabilidad y persistencia de los datos, eliminando el riesgo de pérdida de transacciones en caso de caída del proceso.

3.3.2. Atributos de calidad afectados

Durability La ventana de pérdida de datos que generaba el guardado cada 1-5 segundos desaparece. Si el proceso muere, las transacciones están garantizadas porque Redis las persistió antes de confirmarlas.

Performance Dependiendo del nivel de durabilidad que se configure en Redis, puede agregarse algo de latencia por operación. Es un costo generalmente aceptable frente a la garantía que ofrece, pero es un trade-off que hay que tener en cuenta.

3.4. Persistencia en archivos json

Todo vive en archivos JSON locales. Sin transacciones atómicas, sin estado compatible entre instancias, y con riesgo de corrupción si el archivo falla.

3.4.1. Mejora propuesta

Con la incorporación de Redis, se reemplazó completamente la persistencia en archivos JSON por un almacenamiento centralizado. Esto resuelve los problemas de falta de transacciones atómicas, imposibilidad de compartir estado entre instancias y riesgo de corrupción.

3.4.2. Atributos de calidad afectados

Reliability El riesgo de corrupción de archivos ya no está. Redis maneja la integridad de los datos de forma mucho más robusta que un archivo JSON que puede quedar en estado inconsistente si el proceso muere en medio de una escritura.

Availability El sistema ahora depende de Redis como infraestructura adicional que hay que operar y monitorear. Si Redis no está disponible el sistema no funciona, introduciendo un nuevo punto de falla que antes no existía.

3.4.3. Función `transfer()` mockeada de forma secuencial

```
1 async function transfer(fromAccountId, toAccountId, amount) {  
2   const min = 200;  
3   const max = 400;  
4   return new Promise((resolve) =>  
5     setTimeout(() => resolve(true), Math.random() * (max - min + 1) + min));  
6 }
```

Código 1: Ejemplo de función `transfer()`

La función ignora todos sus parámetros y siempre resuelve `true` tras un delay aleatorio de 200–400 ms.

3.5. Instancia única sin redundancia ni healthcheck

Impacto sobre Availability La API es un Single Point of Failure (SPOF). Un crash del proceso Node.js (error no capturado, OOM, señal del OS), un reinicio por deploy o cualquier excepción no manejada detiene completamente el servicio. Sin `'healthcheck'` declarado, Docker y Nginx no pueden detectar automáticamente que el contenedor dejó de responder. Nginx continuará enrutando requests al upstream caído, retornando 502 Bad Gateway a todos los usuarios.

Impacto sobre User-Perceived Performance Sin distribución de carga entre instancias, todos los requests concurrentes comparten el único event loop de Node.js. Las esperas de `'await transfer()'` (400–800 ms por request) bloquean efectivamente el procesamiento del resto de requests que llegan mientras tanto.

Impacto sobre Scalability Nginx no está configurado para balancear hacia múltiples backends. Aunque se pudieran levantar réplicas (`'docker compose up --scale api=3'`), el upstream hardcodeado a `'exchange-api-1'` ignorará las instancias adicionales. Para habilitar el balanceo se requiere modificar la configuración de Nginx.

3.5.1. Mejora propuesta

Con Redis como estado compartido y Nginx reconfigurado para balancear carga, el **API Server** pasa de ser una instancia única a poder escalar horizontalmente (1..*). Nginx distribuye los requests entre las réplicas disponibles, eliminando el SPOF y permitiendo absorber mayor carga concurrente.

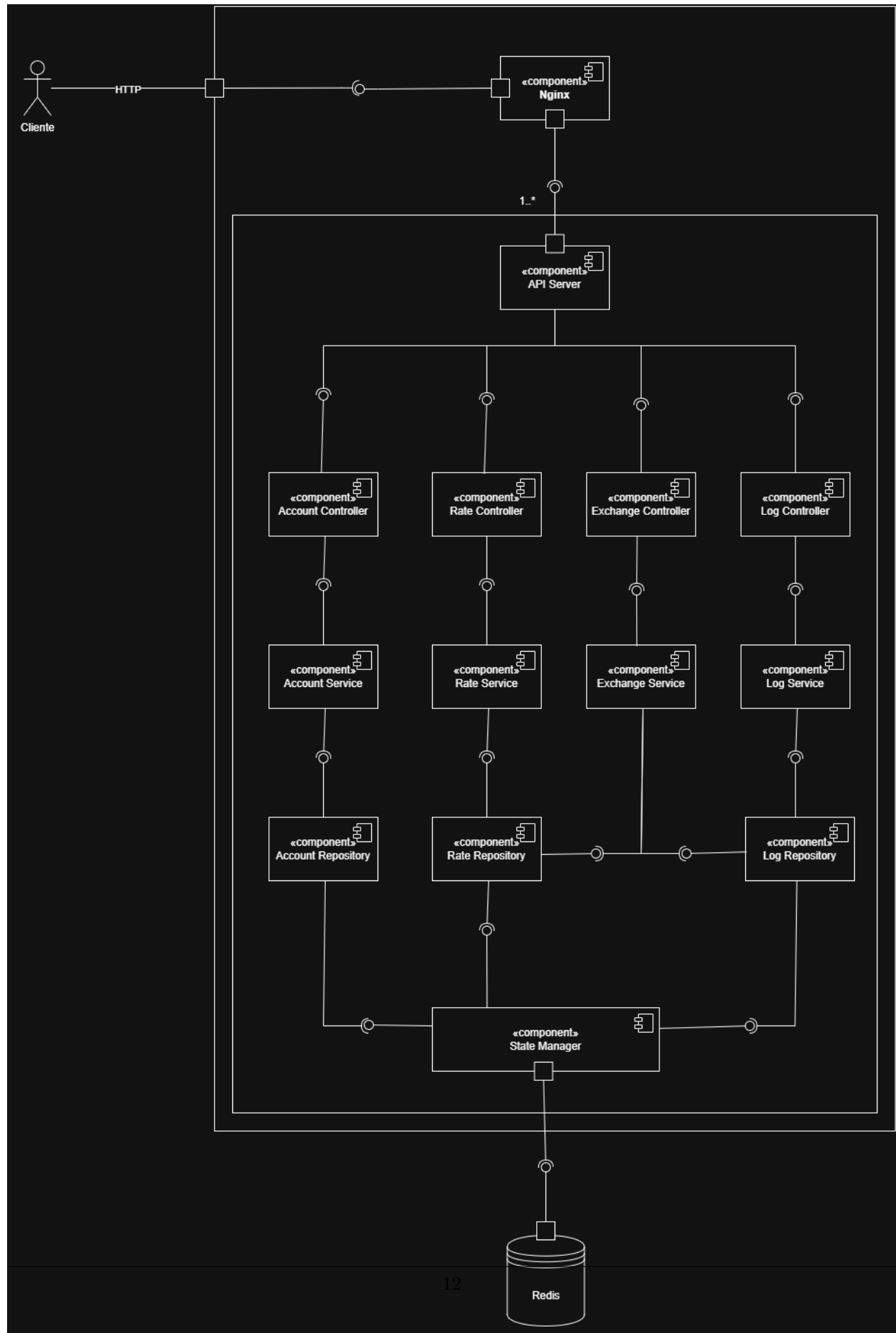


Diagrama 3.4: El único cambio respecto al diagrama anterior es la multiplicidad 1..* entre **Nginx** y la aplicación: ahora pueden existir múltiples instancias, cada una con su propia cadena de Controllers, Services y Repositories, todas compartiendo el mismo estado en Redis.

4. Métricas: Caso Base (Lectura de Rates)

En esta sección se presentan los resultados correspondientes al test de carga base ejecutando el script `perf/rates.yaml`. Las métricas documentadas y la imagen extraída del dashboard de Grafana (`perf/dashboard.json`) retratan el comportamiento del sistema *previo* a la aplicación de cambios relacionados con tácticas de Escalabilidad, pero nos exponen los tiempos crudos de lectura.

```

http.codes.200: ..... 390
http.downloaded_bytes: ..... 42510
http.request_rate: ..... 3/sec
http.requests: ..... 390
http.response_time:
  min: ..... 2
  max: ..... 30
  mean: ..... 3.6
  median: ..... 3
  p95: ..... 5
  p99: ..... 13.1
http.response_time.2xx:
  min: ..... 2
  max: ..... 30
  mean: ..... 3.6
  median: ..... 3
  p95: ..... 5
  p99: ..... 13.1
http.responses: ..... 390
vusers.completed: ..... 390
vusers.created: ..... 390
vusers.created_by_name.Rates (/rates): ..... 390
vusers.failed: ..... 0
vusers.session_length:
  min: ..... 4
  max: ..... 74
  mean: ..... 6.5
  median: ..... 5.3
  p95: ..... 10.5
  p99: ..... 27.9

```

Diagrama 4.1: Métricas obtenidas de la prueba de rendimiento



Imagen 4.1: Métricas obtenidas de la prueba de rendimiento

5. Conclusión (Parcial)

A lo largo del análisis de la arquitectura inicial (*Caso Base*) del *exchange*, se ha constatado que el sistema evidenciaba profundas deficiencias estructurales en sus Atributos de Calidad más críticos, notablemente en relación al eje de **Confiabilidad**, **Escalamiento** e **Integridad Transaccional**.

Los pasos ejecutados hasta el momento (el refactor a una *Layered Architecture*) han mitigado parcialmente las problemáticas de un bajo acoplamiento: si bien al abstraer **controllers**, **services** y **repositories** la **Modificabilidad** aumentó, es importante recalcar que el sistema sigue ligado al disco local y a inyecciones asincrónicas en archivos JSON.

Hacia el horizonte del proyecto, es menester la incorporación de un nodo central de persistencia (Base de datos *ACID* o *NoSQL transaccional*), logrando así transicionar los micro-nodos a verdaderos actores *Stateless*. Sólo entonces, será justificable multiplicar dichos procesos detrás de una capa de *proxy inverso* (Nginx), optimizando su **Escalabilidad Horizontal**.

Por otro lado, se prevé inocular nuevas **Métricas Biológicas de Dominio (Business Metrics)** (por ejemplo, el volumen de criptomonedas y moneda local transaccionados acumulativamente en el tiempo, identificando valores absolutos y transacciones netas). Esta nueva incorporación de observabilidad ampliará significativamente los índices de éxito sobre *Business Intelligence*, y se proyectan visualizar de forma unificada sobre *Grafana*.

[NOTA: Una vez implementadas las métricas de dominio (compras/ventas sumadas por divisa e insertadas en Grafana) y reemplazado el archivo JSON por una BD propiamente dicha, no olvidar reescribir esta sección reemplazando el carácter “Parcial” unificando el aprendizaje total que se dio al resolver los limitantes anteriores.]